03

Sistemas de recomendación



Sistemas de recomendación

Un algoritmo de recomendación es un sistema de toma de decisiones o de puntuación diseñado para sugerir elementos, contenido o acciones relevantes a los usuarios en función de sus preferencias, comportamientos y datos contextuales.

- Filtrado basado en contenido: Sugiere elementos similares a los ya consumidos, basándose en las características del propio contenido
- Filtrado colaborativo: Recomienda elementos analizando los patrones de usuarios con gustos parecidos (el principio de "usuarios que llevaron esto también les gustó aquello").

Pueden basarse en reglas, en similitudes o utilizar aprendizaje automático.

El objetivo principal de los algoritmos de recomendación es mejorar la experiencia del usuario y la toma de decisiones al proporcionar sugerencias personalizadas que reduzcan la sobrecarga de información.

Recomendación basada en contenido

Los filtros basados en contenido tienen el ítem como base de la predicción, en lugar de tener al usuario. Es decir, utiliza las características del ítem (marca, precio, calificaciones, tamaño, categoría, etc.) para hacer las recomendaciones.

Aunque también se puede hacer la recomendación inversa (a qué usuarios les interesa un ítem).

Estos sistemas poseen información de ítems y usuarios.

- El perfil del ítem (o artículo) es la representación del ítem. Consiste en el conjunto de características de un elemento, que pueden ser características internas o metadatos descriptivos.
- El perfil del usuario representa las preferencias y el comportamiento del usuario. Puede consistir en representaciones de aquellos elementos en los que un usuario mostró interés previamente. También consta de datos de usuario de las interacciones pasadas con el sistema (por ejemplo, preferencias, calificaciones, consultas, etc.).

Recomendación basada en contenido

Para el armado de los perfiles se arma una matriz, donde las filas son los ítem y las columnas son los atributos que se pueden medir de cada ítem.

Cada fila (ítem) es un vector en un espacio N-dimensional. Los vectores representantes de ítems similares estarán cerca en el espacio N-dimensional.

	Attr. 1	Attr. 2	Attr. 3	Attr. 4	Attr. 5	Attr. 6	Attr. 7	…	Attr. M
Ítem 1	1	0	1	0	0	0	1	…	0
Ítem 2	0	1	1	0	0	0	0	…	0
Ítem 3	0	0	0	0	1	0	0	…	1
Ítem 4	0	0	0	1	0	1	0	…	1
Ítem 5	0	0	0	0	1	0	0	…	1
…	…	…	…	…	…	…	…	…	…
Ítem N	1	0	1	0	0	1	0	…	0

Sabiendo que ítems les interesa a un usuario, solo hay que buscar los similares (nuevos) y realizar esa recomendación.

Recomendación basada en contenido

Para determinar cuán similares son dos ítems hay que seleccionar un mecanismo de comparación (hay muchas opciones).

Una posibilidad es usar la distancia o similitud coseno, que mide el coseno del ángulo que forman dos vectores.

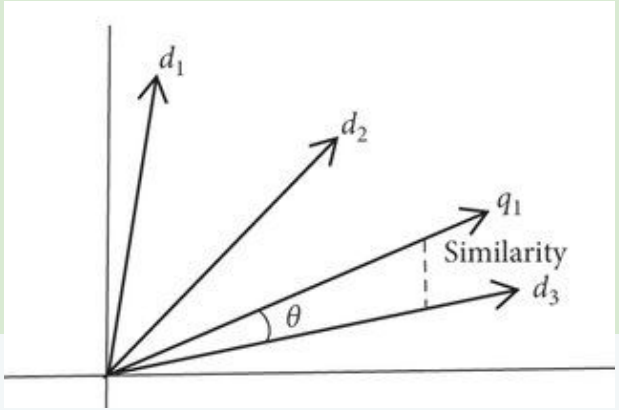
$$cos\theta = \frac{\sum_0^{n-1}(a_i \cdot b_i)}{\sqrt{\sum_0^{n-1} a_i^2} \cdot \sqrt{\sum_0^{n-1} b_i^2}}$$

=1, los vectores apuntan exactamente en la misma dirección (ángulo de 0°).

=0, los vectores son ortogonales (ángulo de 90°).

=-1, los vectores apuntan en direcciones completamente opuestas.

Con lo cual, dado un ítem, solo resta buscar los ítems más parecidos (según la función de similitud usada), y esos serán los ítems a recomendar.



Recomendación basada en contenido

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

item = [[0,1,1,0,0,0,0,1]]  # Una muestra ejemplo, se desea buscar los
                            # 5 ítems más parecidos
c_sim = cosine_similarity(X, item)    # Obtiene la similitud de la
                                      # muestra contra todo el dataset

index_ord = np.argsort(c_sim, axis=0)
top5 = index_ord[-5:].squeeze()       # Obtiene los índices del dataset
                                      # donde están los 5 ítems más
                                      # similares

print(c_sim[top5])            # Imprime los valores de similitud
print(X.iloc[top5])           # Imprime los 5 ítems más similares
```

Filtrado colaborativo

El filtrado colaborativo es una técnica que predice los intereses de un usuario basándose en las valoraciones y el comportamiento de otros usuarios con gustos similares.

Trabajan con una matriz de interacción que relaciona a los usuarios con los ítems (artículos) que han calificado. Los valores de la matriz es la puntuación que le dio el usuario *X* al ítem *Y*. Suele estar “casi” vacía (matriz sparse).

	Ítem 1	Ítem 2	Ítem 3	Ítem 4	Ítem 5	Ítem 6	Ítem 7	…	Ítem M
User 1	5		4				5	…	
User 2		1	2	0				…	
User 3				0	1			…	4
User 4				3		1		…	5
User 5	2	3			1		4	…	1
…	…	…	…	…	…	…	…	…	…
User N	1		5			2		…	

Filtrado colaborativo

El objetivo es “rellenar” los espacios en blanco.

Los valores “recomendados” estarán dentro de la escala del puntaje, luego habrá que seleccionar los de mayor puntaje para realizar la recomendación.

Existen muchas opciones para realizar esta tarea. Una es la conocida como “filtrado colaborativo basado en usuarios con normalización de la media”.

El cálculo del valor de recomendación del ítem *i* para el usuario *u*.

	Ítem 1	Ítem 2	Ítem 3	Ítem 4	…	Ítem M
User 1	5		4		…	
User 2		1	2	0	…	
User 3				0	…	4
User 4				3	…	5
User 5	2	3			…	1
…	…	…	…	…	…	…
User N	1		5		…	

Se calculan las similitudes del usuario *u* contra todos los que puntuaron el ítem *i*.

Similitud
1.72
3.01
4.32

Filtrado colaborativo

Cálculo del valor de recomendación del ítem *i* para el usuario *u*.

	Ítem 1	Ítem 2	Ítem 3	Ítem 4	…	Ítem M	Similitud
User 1	5		4		…		1.72
User 2		1	2	0	…		…
User 3	**1.35**	**3.56**	**4.12**	0	…	4	…
User 4				3	…	5	…
User 5	2	3			…	1	3.01
…	…	…	…	…	…	…	…
User N	1		5		…		4.32

$$V(u,i) = R_u + \frac{\sum_o Sim(u,o).(R_{o,i} - R_v)}{\sum_o Sim(u,o)}$$

Sim(u,o) es la similitud entre los usuarios *u* y *o*.
R_u es el promedio de los valores puntuados.
$R_{o,i}$ es el ranking del usuario *o* al ítem *i*.

La principal desventaja de esta técnica es que no resulta muy exacta para usuarios nuevos, o que tienen pocas puntuaciones (problema del arranque en frío).

Filtrado colaborativo

```
# X_nan es un dataframe con "nan" para los ítems no puntuados.
usuario_id = 12    # El usuario a quien recomendar ítems
item_i = 108       # El ítem a calcular su score
n_cols = X_nan.shape[1]     # El número de atributos del dataset

Ru = X_nan.iloc[usuario_id].mean()

numerador = 0.0; denominador = 0.0
for usuario_v in X_nan.index:    # Iterar sobre todos los usuarios
     if usuario_v == usuario_id:
          continue
     if not np.isnan(puntua):  # Si tiene una calificación válida
          # Sumar la distancia entre el usuario_id y usuario_v (a continuación...)

# El score para (usuario_id, item_i)
if denominador == 0:
     score = Ru
else:
     score = Ru + (numerador / denominador)
```

Filtrado colaborativo

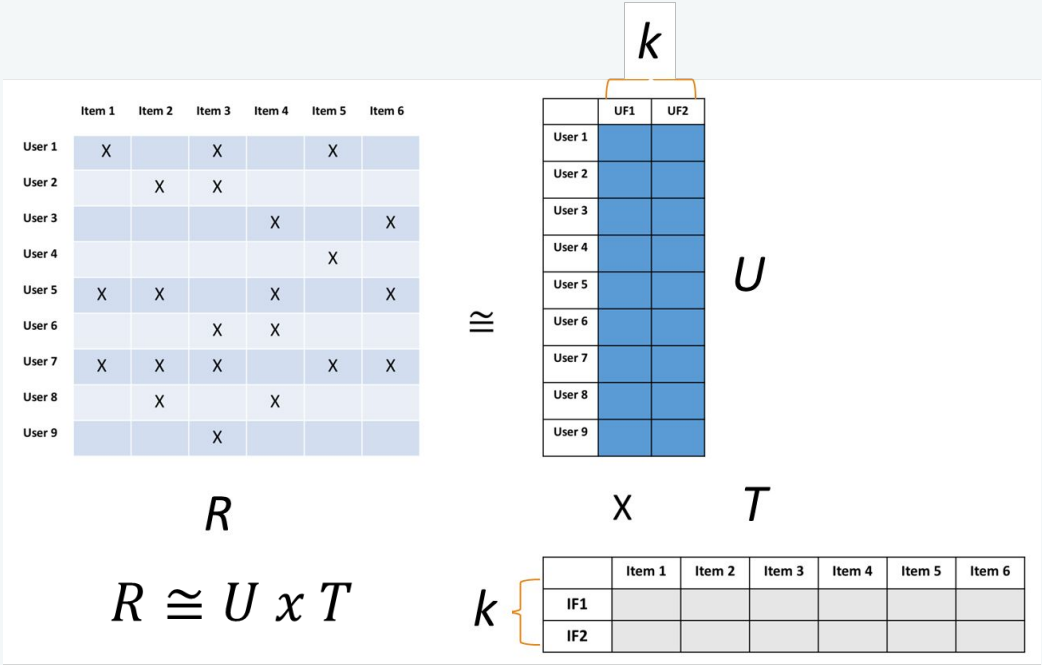
```
        # if ...
            # Sumar la distancia entre el usuario_id y usuario_v
            sim_u_v_num = 0; sim_u_v_den_a = 0; sim_u_v_den_b = 0
            for i in range(n_cols):
                a = X_nan.iloc[usuario_id, i]
                b = X_nan.iloc[usuario_v, i]
                if not np.isnan(a) and not np.isnan(b):
                    sim_u_v_num+= a * b
                    sim_u_v_den_a+= a * a
                    sim_u_v_den_b+= b * b
            den = (np.sqrt(sim_u_v_den_a) * np.sqrt(sim_u_v_den_b))
            if den != 0:
                sim_u_v = sim_u_v_num / den
                Rv = X_nan.iloc[usuario_v].mean()

                numerador += sim_u_v * (puntua - Rv)
                denominador += abs(sim_u_v)

    # if ...
```

ALS (Alternating least square)

El algoritmo ALS (Alternating least square) intenta descubrir dos matrices (T y U), tal que el producto de ellas sea la matriz de recomendación R.

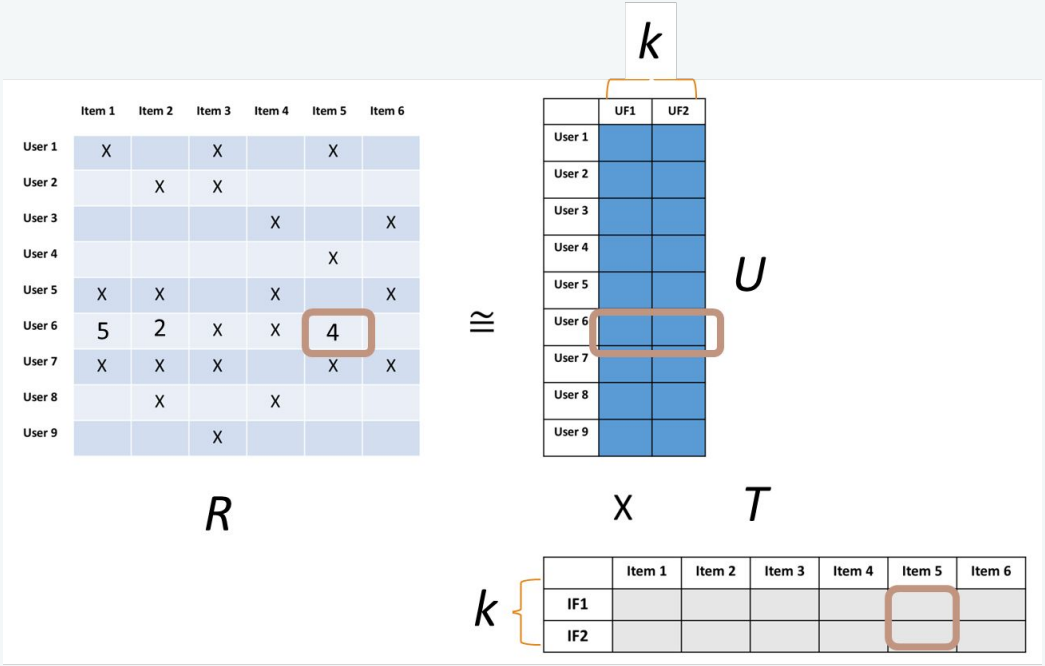


$$U_j = \left[\sum_{i;r_{ji}>0} \left(T_i T_i^t\right) + \lambda I_k\right]^{-1} \sum_{i;r_{ji}>0} \left(R_{ji} T_i\right)$$

$$T_j = \left[\sum_{i;r_{ji}>0} \left(U_i U_i^t\right) + \lambda I_k\right]^{-1} \sum_{i;r_{ji}>0} \left(R_{ji} U_i\right)$$

ALS (Alternating least square)

Las recomendaciones salen de multiplicar la fila de la matriz U correspondiente al usuario, con la columna de la matriz T para un determinado ítem.

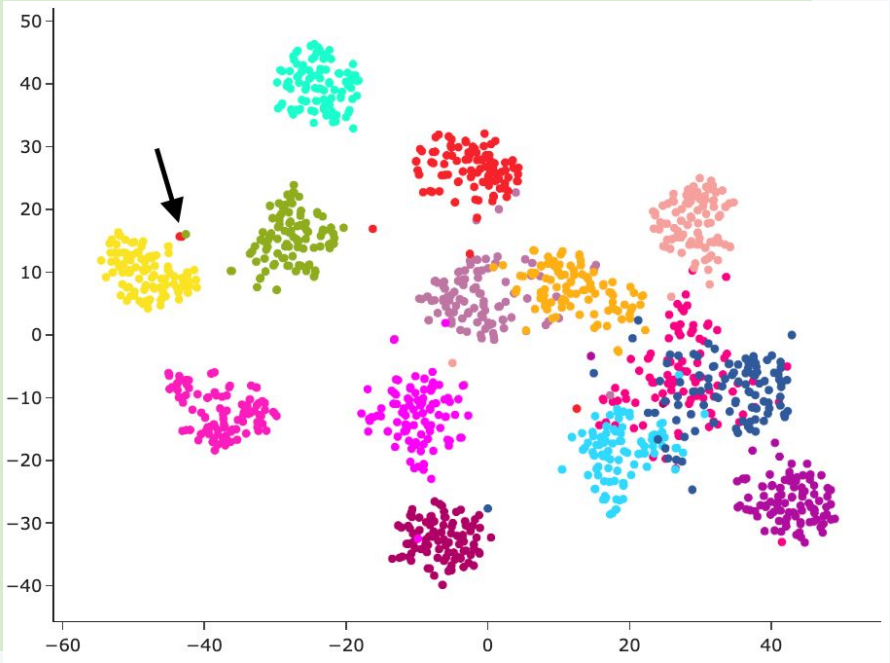


Filtrado colaborativo utilizando embeddings

Alternativas más modernas utilizan el concepto de vector de embeddings. A cada usuario y a cada ítem se le calcula su vector e embedding usando modelos más complejos (deep learning).

Los vectores de usuarios con gustos similares, o ítems con características similares, serán cercanos en el espacio de los vectores de embeddings (dimensionalidad alta: 128, 256, 512, o más).

Luego, utilizando alguna métrica de similitud, se pueden realizar las recomendaciones buscando los ítems de usuarios similares, o de ítems con características similares a las consumidas por un usuario.



Actividades

El dataset “movies” tiene el nombre de casi 4000 películas y los géneros a los cuales pertenece cada una.

El formato del archivo es:

movie_id::título y año::generos

Los géneros están separados por “|”

3. Use la base de datos “movies” para realizar recomendaciones basadas en contenido usando la similitud coseno.

   Elija una película entre las 3883 que tiene el dataset y devuelva el top 10 de las películas a recomendar.

   Use:

   ```
   X_matrix = X['generos'].str.get_dummies(sep='|')
   ```

   Para convertir el dataset “movies” en la matriz que se necesita.

Actividades

El dataset “ratings” tiene la puntuación (de 1 a 5) otorgada a películas por los usuarios que las vieron.

El formato del archivo es:

user_id::movie_id::puntuación::timestamp

4. Use la base de datos “ratings” para realizar recomendaciones por filtrado colaborativo basado en usuarios con normalización de la media.

   Elija un usuario entre los 6040 que tiene el dataset y calcule la recomendación para cada película no vista.

   Use:

```
X_matrix = pd.pivot_table(X, values='puntuación', index='user_id',
                          columns='movie_id').fillna(np.nan)
```

   Para convertir el dataset “ratings” en la matriz que se necesita.

RESUMEN

- Técnicas de balance de clases
- Selección de características
- Sistemas de recomendación

